C++ Moderne — Norme C++20

Support de Cours Professionnel

Profil cible : Développeur C / Python migrant vers le C++ moderne
Norme couverte : C++20
Niveau : Intermédiaire → Avancé

PARTIE 1 — RÉVISION ET CONSOLIDATION

Chapitre 1 — Espaces de Noms (Namespaces)

1.1 Pourquoi les namespaces existent

En C, l'espace de nommage global est **unique et plat**. Si deux bibliothèques définissent chacune une fonction `init()`, le linker se retrouve avec un conflit de symboles. La solution C était les préfixes : `SDL_Init()`, `GL_Init()`, etc. C'est du nommage manuel, fragile et verbeux.

C++ résout ce problème au niveau du langage avec les **namespaces** : des conteneurs logiques qui créent des portées distinctes pour les identifiants.

```cpp
// Bibliothèque A
namespace audio {
    void init();
    void shutdown();
}

// Bibliothèque B
namespace video {
    void init();
    void shutdown();
}

// Pas de conflit : audio::init() != video::init()
int main() {
    audio::init();
    video::init();
    return 0;
}
```

1.2 L'opérateur de résolution de portée `::`

L'opérateur `::` est le **navigateur de portée** du C++. Il indique explicitement dans quel namespace (ou quelle classe) chercher un identifiant.

```cpp
namespace corp::engine {          // Namespace imbriqué (C++17+)
    namespace physics {
        float gravity = 9.81f;
    }
}

float g = corp::engine::physics::gravity;  // Chemin complet
```

L'opérateur `::` seul (sans préfixe) désigne le **namespace global** — utile pour lever une ambiguïté :

```cpp
int value = 42;

namespace local {
    int value = 0;
    void print() {
        // ::value => variable globale
        // value   => local::value
    }
}
```

1.3 Pourquoi interdire `using namespace std;`

`using namespace std;` **injecte** la totalité de la bibliothèque standard dans votre portée courante. La stdlib C++ contient des centaines d'identifiants : `distance`, `count`, `fill`, `remove`, `find`, `sort`, `swap`, `min`, `max`... Ces noms sont extrêmement génériques.

```cpp
// Piège classique en entreprise
#include <algorithm>
using namespace std;

int distance(Point a, Point b) { ... } // ERREUR: conflict avec std::distance
```

La règle en production :

- `using namespace` est **interdit dans les headers** (.h / .hpp) : il contaminerait chaque fichier qui l'inclut.
- Dans les fichiers .cpp, on tolère au maximum un `using` **local** dans le corps d'une fonction, pas au niveau du fichier.
- La bonne pratique industrielle : toujours qualifier explicitement (`std::vector`, `std::cout`).

```cpp
// Compromis acceptable : alias de namespace
namespace fs = std::filesystem;
fs::path p = "/tmp/data";
```

1.4 Namespaces anonymes

Un namespace sans nom est **visible uniquement dans son unité de compilation**. C'est le remplaçant C++ du mot-clé `static` au niveau fichier en C :

```cpp
// En C : visibilité fichier via static
static int counter = 0;

// En C++ moderne : namespace anonyme
namespace {
    int counter = 0;    // Inaccessible depuis d'autres .cpp
    void helper() { }   // Idem
}
```

> **Cahier des Charges — Exercice 1 : Système de Configuration**
> **Contexte métier :** Vous intégrez une équipe qui développe un moteur de jeu. Votre première tâche est de poser les bases de l'architecture de configuration du moteur.
> **Règles métier à implémenter :**
> 1. Le projet doit être organisé en au moins deux espaces de noms distincts : un pour le moteur (`engine`) et un sous-espace de noms dédié à la configuration (`engine::config`).
> 2. L'espace `engine::config` doit exposer des constantes nommées pour les paramètres suivants : la résolution cible (largeur et hauteur), le nombre d'images par seconde cible, et le nom de l'application.
> 3. L'espace `engine` doit exposer une fonction d'initialisation qui affiche un résumé de la configuration sur la sortie standard. Aucun `using namespace` ne doit apparaître dans

le code.

4. Le `main` doit se contenter d'appeler la fonction d'initialisation du moteur, sans accéder directement aux constantes de configuration.
5. Toute fonction utilitaire interne qui ne doit pas être visible à l'extérieur du fichier `.cpp` doit être correctement encapsulée.

Chapitre 2 — Les Références

2.1 Référence vs Pointeur : la différence fondamentale

Un **pointeur** en C est une variable qui stocke une **adresse mémoire**. Il peut être nul (`NULL`/`nullptr`), réaffecté pour pointer sur autre chose, et doit être déréférencé manuellement (`*ptr`).

Une **référence** C++ est un **alias** vers une variable existante. Sous le capot, le compilateur l'implémente généralement comme un pointeur constant (non réaffectable), mais il vous libère de la syntaxe de déréférencement.

```cpp
// En C
void increment_c(int *p) {
    if (p == NULL) return;  // Garde obligatoire
    *p += 1;                // Déréférencement explicite
}

// En C++ avec référence
void increment_cpp(int &val) {
    // val ne peut pas être nul, pas de garde nécessaire
    // pas de déréférencement : syntaxe directe
    val += 1;
}
```

Propriétés garanties d'une référence :

- **Non nulle** : une référence doit obligatoirement être initialisée à la déclaration.
- **Non réaffectable** : une fois liée à une variable, elle le reste pour toujours.
- **Pas de syntaxe de déréférencement** : on l'utilise comme la variable elle-même.

2.2 La référence constante `const &`

Le passage par valeur implique une **copie**. Pour un `int`, le coût est négligeable. Pour un `std::vector` de 10 millions d'éléments, la copie est catastrophique. Le passage par `const &` est le mécanisme clé de performance en C++ :

```cpp
// Copie : O(n) en mémoire et en temps
void process_bad(std::vector<int> data) { ... }

// Référence const : O(1), aucune copie, modification impossible
void process_good(const std::vector<int> &data) { ... }
```

Quand utiliser quoi :

Cas d'usage	Mécanisme	Raison
Petit type (int, float, char)	Valeur	Copie moins chère qu'une indirection
Grand objet, lecture seule	`const &`	Zéro copie, protégé contre modification
Grand objet, modification souhaitée	`&`	Zéro copie, modification de l'original
Ownership transfer (C++11+)	`&&` (move)	Déplacement de ressources

2.3 Durée de vie et dangling references

Le compilateur peut **étendre la durée de vie** d'un temporaire lié à une `const &` :

```cpp
const std::string &ref = std::string("temporaire");
// Le temporaire survit tant que ref est en scope : valide en C++
```

Mais la référence ne peut pas **outlive** sa source :

```cpp
int &dangling_ref() {
    int local = 42;
    return local;  // UNDEFINED BEHAVIOR : local est détruit au retour
}
```

> **Cahier des Charges — Exercice 2 : Catalogue de Produits**
> **Contexte métier :** Vous développez un module de gestion de catalogue pour une boutique en ligne.
> **Règles métier à implémenter :**
> 1. Dans l'espace de noms `engine::config` de l'exercice précédent, ajoutez une constante représentant le nombre maximum de produits dans le catalogue.

2. Dans un espace de noms `catalog`, définissez une structure de données représentant un produit avec : un identifiant entier, un nom textuel, et un prix décimal.
3. Implémentez une fonction qui reçoit une collection de produits **sans la copier** et retourne **par référence constante** le produit dont le prix est le plus élevé.
4. Implémentez une fonction qui reçoit un produit **par référence modifiable** et applique une remise (en pourcentage) sur son prix.
5. Le programme principal doit créer une collection d'au moins 5 produits, trouver le plus cher, afficher son nom et son prix, puis lui appliquer une remise de 10% et afficher le nouveau prix. Aucun `using namespace` autorisé.

Chapitre 3 — Initialisation Moderne

3.1 Le problème de l'initialisation en C++03

Avant C++11, il existait une multitude de syntaxes d'initialisation inconsistantes selon le type de l'objet :

```cpp
int a = 5;            // Copie d'initialisation
int b(5);             // Initialisation directe
int arr[] = {1,2,3}; // Initialisation d'agrégat
Widget w;             // Constructeur par défaut
Widget w2(arg);       // Constructeur avec argument
```

Ce zoo syntaxique créait des ambiguïtés, notamment la célèbre **Most Vexing Parse** :

```cpp
Widget w();  // PIÈGE : déclaration d'une fonction, pas d'un objet !
```

3.2 L'initialisation uniforme avec `{}`

C++11 introduit une syntaxe universelle avec les accolades, applicable à **tout type** :

```cpp
```

```
int a{5};
double pi{3.14159};
std::string s{"hello"};
std::vector<int> v{1, 2, 3, 4, 5};

struct Point { int x, y; };
Point p{10, 20};          // Initialisation d'agrégat

// Plus de Most Vexing Parse
Widget w{};               // Toujours un objet, jamais une déclaration de fonction
```

3.3 Narrowing detection

L'avantage majeur de `{}` : il **détecte les conversions avec perte de données** à la compilation, là où `=` ou `()` les acceptent silencieusement :

```cpp
int x = 3.7;     // Silencieux : x vaut 3, perte de 0.7
int y(3.7);      // Silencieux aussi

int z{3.7};      // ERREUR DE COMPILATION : narrowing conversion interdit
```

En entreprise, cette propriété est précieuse : des bugs de troncature numérique sont détectés au build plutôt qu'en production.

3.4 `std::initializer_list`

Quand vous écrivez `{1, 2, 3}` pour initialiser un objet, le compilateur crée implicitement un `std::initializer_list<int>`. Les conteneurs de la stdlib exploitent cela via des constructeurs dédiés. Vous pouvez faire de même dans vos propres classes.

> **Cahier des Charges — Exercice 3 : Registre de Sessions**
> **Contexte métier :** Le moteur de jeu doit gérer des sessions de jeu avec leurs paramètres initiaux.
> **Règles métier à implémenter :**
> 1. Dans l'espace de noms `engine`, définissez une structure `GameSession` avec : un identifiant de session, un nom de joueur, un niveau de départ (avec valeur par défaut à 1), et un score initial (avec valeur par défaut à 0).
> 2. Toutes les instanciations de `GameSession` dans le code doivent utiliser exclusivement la syntaxe d'initialisation uniforme.
> 3. Implémentez une fonction dans `engine` qui prend une liste de sessions (sans copie) et retourne par référence constante la session avec le score le plus élevé. Si la liste est vide,

une exception `std::invalid_argument` doit être levée.

4. Dans le `main`, créez une collection d'au moins 4 sessions en utilisant l'initialisation uniforme, appelez la fonction de recherche dans un bloc `try/catch`, et affichez le nom du joueur gagnant. Aucun `using namespace` autorisé.

Chapitre 4 — Structures et Classes (POO)

4.1 `struct` vs `class` : une seule différence

En C++, `struct` et `class` sont **quasi-identiques**. La seule différence est la visibilité par défaut des membres :

- `struct` : membres `public` par défaut.
- `class` : membres `private` par défaut.

En pratique industrielle : `struct` pour les agrégats de données simples (type "plain old data"), `class` pour les entités avec comportement et invariants à protéger.

4.2 Le pointeur `this` implicite

En C, pour simuler des méthodes, on passait explicitement un pointeur vers la structure :

```c
// Simulation de méthode en C
typedef struct { int x, y; } Point;
float distance_c(const Point *self, const Point *other) {
    // ...
}
distance_c(&p1, &p2);
```

En C++, ce pointeur `self` existe toujours mais est **implicite** et s'appelle `this`. Chaque méthode non-statique reçoit automatiquement un pointeur `this` vers l'instance courante.

```cpp
```

```
class Point {
    int x_, y_;
public:
    // Le compilateur passe implicitement Point* this
    float distance(const Point &other) const {
        // this->x_ == x_ (le compilateur résout automatiquement)
        int dx = this->x_ - other.x_;
        int dy = this->y_ - other.y_;
        return std::sqrt(dx*dx + dy*dy);
    }
};
```

Le `const` après les parenthèses d'une méthode signifie que `this` est un `const Point*` : la méthode garantit de ne pas modifier l'objet.

4.3 Encapsulation et invariants

L'encapsulation ne sert pas à "cacher" par principe — elle sert à **maintenir des invariants**. Un invariant est une propriété qui doit toujours être vraie sur l'objet.

```cpp
class BoundedCounter {
    int value_;
    int max_;

public:
    BoundedCounter(int max) : value_{0}, max_{max} {}

    // L'invariant : value_ est toujours dans [0, max_]
    // Sans encapsulation, n'importe qui peut faire counter.value_ = -99
    void increment() {
        if (value_ < max_) ++value_;
    }
    int value() const { return value_; }
};
```

> **Cahier des Charges — Exercice 4 : Inventaire du Joueur**
> **Contexte métier :** Le moteur de jeu doit gérer l'inventaire d'un personnage joueur.
> **Règles métier à implémenter :**
> 1. Dans `engine`, créez une classe `Inventory` qui encapsule une collection d'items (chaque item a un nom et un poids en grammes).
> 2. La classe doit maintenir l'invariant suivant : le poids total de l'inventaire ne peut pas dépasser une capacité maximale définie à la construction. Toute tentative d'ajout qui

dépasserait la capacité doit lever une exception `std::runtime_error`.

3. La classe doit exposer des méthodes pour : ajouter un item, obtenir le nombre d'items, obtenir le poids total actuel, et afficher le contenu de l'inventaire.
4. Toutes les méthodes qui ne modifient pas l'état de l'objet doivent être marquées `const`.
5. Le `main` doit créer un inventaire, tenter d'y ajouter plusieurs items (dont un qui dépasse la capacité), gérer l'exception, et afficher l'état final. Utiliser l'initialisation uniforme partout. Aucun `using namespace`.

Chapitre 5 — Les Constructeurs

5.1 La Member Initializer List (MIL)

La syntaxe `: membre{valeur}` après la signature du constructeur n'est pas optionnelle pour les cas critiques — c'est une **construction directe** des membres. Sans MIL, les membres sont **construits par défaut puis réassignés**, ce qui est au mieux un gaspillage, au pire une erreur de compilation.

```cpp
class Connection {
    const std::string host_;  // const : DOIT être initialisé via MIL
    int port_;
    Socket socket_;           // Pas de constructeur par défaut : DOIT être via MIL

public:
    // Correct : construction directe via MIL
    Connection(std::string host, int port)
        : host_{std::move(host)}, port_{port}, socket_{host_, port_}
    {}

    // INCORRECT : host_ est const, on ne peut pas l'assigner dans le corps
    // Connection(std::string host, int port) {
    //     host_ = host;  // ERREUR DE COMPILATION
    // }
};
```

Ordre d'initialisation : Les membres sont initialisés dans l'**ordre de leur déclaration dans la classe**, pas dans l'ordre de la MIL. Déclarez-les toujours dans le bon ordre.

5.2 Valeurs par défaut et surcharge

```cpp
```

```cpp
class Server {
    std::string address_;
    int port_;
    int timeout_ms_;

public:
    // Valeurs par défaut : permettent d'omettre des arguments
    Server(std::string addr, int port = 8080, int timeout = 5000)
        : address_{std::move(addr)}, port_{port}, timeout_ms_{timeout}
    {}
};

Server s1{"localhost"};            // port=8080, timeout=5000
Server s2{"prod.server.com", 443}; // timeout=5000
Server s3{"db.local", 5432, 1000};
```

5.3 Délégation de constructeurs (C++11)

Un constructeur peut en appeler un autre pour éviter la duplication :

```cpp
class Rectangle {
    int w_, h_;
public:
    Rectangle(int w, int h) : w_{w}, h_{h} {}
    Rectangle(int side) : Rectangle{side, side} {}  // Délégation
    Rectangle() : Rectangle{1} {}                    // Chaîne de délégation
};
```

5.4 `explicit` : prévenir les conversions implicites

```cpp
class Radius {
    double value_;
public:
    explicit Radius(double v) : value_{v} {}
};

void draw_circle(Radius r) { ... }

draw_circle(5.0);        // ERREUR : conversion implicite bloquée par explicit
draw_circle(Radius{5.0}); // OK : conversion explicite
```

Sans `explicit`, le compilateur convertirait silencieusement n'importe quel `double` en `Radius`, source de bugs subtils.

> **Cahier des Charges — Exercice 5 : Personnage de Jeu**
> **Contexte métier :** Le moteur a besoin d'une classe représentant un personnage jouable.
> **Règles métier à implémenter :**
>
> 1. Dans `engine`, créez une classe `Character` avec les attributs suivants : nom (non modifiable après construction), points de vie (PV) maximum, PV actuels, niveau, et points d'expérience.
> 2. Le constructeur principal prend un nom et des PV maximum. Le niveau démarre à 1 et les PV actuels sont égaux aux PV maximum. Toutes les initialisations passent par la MIL.
> 3. Fournissez un second constructeur qui accepte également un niveau de départ (entre 1 et 100 ; lever une exception si hors limites). Utilisez la délégation de constructeurs pour éviter la duplication.
> 4. Implémentez les méthodes : `takeDamage(int)` (réduit les PV sans les faire passer sous 0), `heal(int)` (augmente les PV sans dépasser le maximum), `gainExperience(int)` (tous les 100 XP, le niveau augmente et les PV sont restaurés au maximum), et `isAlive() const`.
> 5. Le `main` crée au moins deux personnages (dont un via le constructeur avec niveau), simule des échanges de dégâts, des soins et des gains d'XP, et affiche l'état après chaque action significative. Tout le code utilise l'initialisation uniforme et aucun `using namespace`.

Chapitre 6 — Itérations Modernes

6.1 `auto` : inférence de type

`auto` demande au compilateur de **déduire le type** d'une variable à partir de son initialiseur. C'est une commodité syntaxique, pas de la dynamicité : le type est résolu **à la compilation**, pas à l'exécution.

```cpp
```

```cpp
auto i = 42;                // int
auto d = 3.14;              // double
auto s = std::string{"hello"}; // std::string

// Cas où auto est indispensable : types verbeux
std::map<std::string, std::vector<int>> registry;
auto it = registry.begin();  // vs std::map<std::string, std::vector<int>>::iterator
```

Règles d'inférence importantes :

- `auto` déduit **sans références ni const** : `auto x = ref;` donne un `int`, pas un `int&`.
- `auto &` préserve la référence.
- `const auto &` préserve const et référence.

```cpp
std::string name = "engine";
auto a = name;           // Copie : std::string
auto &b = name;          // Référence : std::string&
const auto &c = name;    // Référence const : const std::string&
```

6.2 Range-based for

La boucle `for (auto elem : container)` itère sur tous les éléments d'un range. Sous le capot, le compilateur génère du code équivalent à une boucle sur itérateurs :

```cpp
// Ce que vous écrivez
for (auto &elem : collection) { ... }

// Ce que le compilateur génère (approximation)
{
    auto __begin = collection.begin();
    auto __end = collection.end();
    for (; __begin != __end; ++__begin) {
        auto &elem = *__begin;
        ...
    }
}
```

Trois usages distincts :

```cpp
```

```
std::vector<std::string> names{"Alice", "Bob", "Charlie"};

// 1. Copie : modification locale uniquement
for (auto name : names) { name += "!"; }  // names inchangé

// 2. Référence : modification de l'original
for (auto &name : names) { name += "!"; }  // names modifié

// 3. Référence const : lecture sans copie (pattern le plus courant)
for (const auto &name : names) {
    std::cout << name << "\n";
}
```

Pour vos propres types, le range-based for fonctionne si le type expose `begin()` et `end()`.

> **Cahier des Charges — Exercice 6 : Groupe de Personnages**
> **Contexte métier :** Le moteur doit pouvoir gérer des groupes de personnages (équipe de joueurs ou vague d'ennemis).
> **Règles métier à implémenter :**
> 1. Dans `engine`, créez une classe `Party` (groupe) qui encapsule une collection de `Character` (du chapitre précédent).
> 2. La classe expose des méthodes pour : ajouter un personnage au groupe, obtenir le nombre de membres, et obtenir une référence constante vers un membre par son index (lever une exception si hors limites).
> 3. Implémentez une méthode `applyDamageToAll(int damage)` qui applique des dégâts à tous les membres du groupe en utilisant une boucle range-based for avec référence modifiable.
> 4. Implémentez une méthode `getAliveCount() const` qui compte les personnages en vie, en utilisant une boucle range-based for avec référence constante.
> 5. Implémentez une méthode `getLevelAverage() const` qui retourne le niveau moyen du groupe. Utilisez `auto` pour le type de l'itérateur ou du résultat intermédiaire.
> 6. Le `main` crée deux groupes, leur applique des dégâts, affiche les statistiques, et gère toutes les exceptions. Initialisation uniforme et zéro `using namespace`.

Chapitre 7 — Conteneurs Standards : `std::vector`

7.1 Architecture interne de `std::vector`

`std::vector<T>` est un **tableau dynamique** : il maintient un bloc de mémoire contiguë (comme

un tableau C) mais gère automatiquement sa taille. Trois pointeurs internes le définissent :

```
[ data_ ]--->[  e0  |  e1  |  e2  |  e3  | (vide) | (vide) ]
              ^                   ^                ^
           begin()             size()          capacity()
```

- `size()` : nombre d'éléments actuellement stockés.
- `capacity()` : taille du bloc alloué (éléments maximum sans réallocation).
- Quand `size == capacity` et qu'on ajoute un élément, le vector **alloue un nouveau bloc** (typiquement 2x la capacité), **copie/déplace** tous les éléments, puis libère l'ancien bloc.

Cette réallocation **invalide tous les pointeurs et iterateurs** vers les éléments. C'est un bug courant :

```cpp
std::vector<int> v{1, 2, 3};
int *ptr = &v[0];   // Pointeur vers le premier élément
v.push_back(4);     // Peut déclencher une réallocation
*ptr = 99;          // UNDEFINED BEHAVIOR : ptr peut être invalide
```

7.2 `reserve` : optimiser les performances

Si vous connaissez à l'avance le nombre d'éléments, `reserve` pré-alloue la mémoire et évite les réallocations successives :

```cpp
std::vector<int> v;
v.reserve(1000);     // Un seul malloc, pas de réallocation
for (int i = 0; i < 1000; ++i) {
    v.push_back(i);  // Jamais de réallocation
}
```

7.3 Opérations clés

```cpp
```

```
std::vector<std::string> v;

v.push_back("hello");          // Copie à la fin
v.emplace_back("world");       // Construction directe à la fin (plus efficace)
v.pop_back();                  // Supprime le dernier élément
v.insert(v.begin(), "start");  // Insertion en position (O(n) !)
v.erase(v.begin() + 1);        // Suppression en position (O(n) !)

v.front();  // Premier élément
v.back();   // Dernier élément
v[i];       // Accès sans vérification de bornes (UB si hors limites)
v.at(i);    // Accès avec vérification (lève std::out_of_range)
```

7.4 Contiguïté mémoire et interopérabilité C

La contiguïté est garantie par le standard. On peut passer un `std::vector` à une API C :

```cpp
std::vector<float> data{1.0f, 2.0f, 3.0f};
// data.data() retourne float*, pointant sur le premier élément
c_api_function(data.data(), data.size());
```

Cahier des Charges — Exercice 7 : Système de Log du Moteur
Contexte métier : Le moteur a besoin d'un système de journalisation pour tracer les événements de la session de jeu.
Règles métier à implémenter :

1. Dans `engine`, créez une classe `EventLog` qui stocke une collection d'entrées de log. Chaque entrée contient : un horodatage (entier simulé, incrémenté à chaque entrée), une sévérité (enum class : INFO, WARNING, ERROR), et un message textuel.
2. La classe doit pré-allouer une capacité initiale raisonnable à la construction pour éviter les réallocations sur les premières entrées.
3. La classe expose des méthodes pour : ajouter une entrée, obtenir le nombre total d'entrées, et afficher toutes les entrées formatées.
4. Implémentez une méthode `getEntriesBySeverity(Severity) const` qui retourne un **nouveau** `std::vector` contenant les entrées correspondant au niveau de sévérité demandé.
5. La classe `Character` du chapitre 5 doit être modifiée pour accepter une référence vers un `EventLog` à la construction et l'utiliser pour journaliser les événements significatifs (dégâts reçus, soins, level-up).

> 6. Le `main` orchestre une session complète : création d'un log, d'un groupe de personnages partageant ce log, simulation de combat, puis affichage des logs filtrés par sévérité.

Chapitre 8 — Gestion des Erreurs : Exceptions

8.1 Les exceptions vs les codes de retour

En C, la gestion d'erreur passe par des codes de retour. Ce mécanisme a des défauts structurels : le code d'erreur peut être ignoré silencieusement, il "pollue" la signature des fonctions, et il rend le chemin nominal difficile à lire.

```c
// C : code de retour
int result = parse_config(path, &config);
if (result == ERR_FILE_NOT_FOUND) { ... }
else if (result == ERR_INVALID_FORMAT) { ... }
// Si j'oublie le if : l'erreur est silencieusement ignorée
```

Les exceptions C++ **séparent le chemin nominal du chemin d'erreur** et **ne peuvent pas être ignorées** : si elles ne sont pas attrapées, elles remontent la pile d'appels et terminent le programme.

8.2 La hiérarchie `std::exception`

```
std::exception
├── std::logic_error          (erreurs détectables à la conception)
│   ├── std::invalid_argument  (argument invalide)
│   ├── std::out_of_range      (index hors bornes)
│   └── std::length_error      (longueur invalide)
└── std::runtime_error        (erreurs détectables à l'exécution)
    ├── std::overflow_error
    ├── std::underflow_error
    └── std::system_error
```

Quand utiliser quoi :

- `std::invalid_argument` : précondition violée (argument null, valeur négative attendue positive…).
- `std::out_of_range` : accès hors des limites d'une collection.
- `std::runtime_error` : erreur survenant à l'exécution (fichier manquant, connexion refusée…).

8.3 Stack Unwinding

Quand une exception est levée, le C++ **déroule la pile** (*stack unwinding*) : il détruit, dans l'ordre inverse de leur création, tous les objets locaux des frames traversées. C'est ce mécanisme qui garantit que les destructeurs sont toujours appelés, même en cas d'erreur.

```cpp
void risky_function() {
    std::vector<int> data{1, 2, 3}; // Construit
    ManagedResource res{};           // Construit
    throw std::runtime_error{"oops"};
    // res est détruit, puis data est détruit → pas de fuite
}
```

8.4 Bonnes pratiques

```cpp
// 1. Attraper par référence const, jamais par valeur
try { ... }
catch (const std::exception &e) {    // Correct
    std::cerr << e.what() << "\n";
}

// 2. Catch général en dernier recours
catch (...) {
    // Attrape tout, mais perd l'information sur l'erreur
}

// 3. noexcept : garantie qu'une fonction ne lève pas
double safe_sqrt(double x) noexcept {
    return x >= 0 ? std::sqrt(x) : 0.0;
}
```

> **Cahier des Charges — Exercice 8 : Sauvegarde de Session (Exercice de Synthèse Partie 1)**
> **Contexte métier :** Le moteur doit pouvoir sauvegarder et charger l'état d'une session de jeu. Cet exercice est la **synthèse de toute la Partie 1**.
> **Règles métier à implémenter :**
> 1. Dans `engine`, créez une classe `SaveManager` responsable de la sérialisation et désérialisation de l'état d'une `Party`.
> 2. La méthode `save(const Party&, const std::string& filename)` doit simuler une sauvegarde en écrivant les données de chaque personnage dans un `std::vector`

d'enregistrements textuels (simuler l'écriture disque par un affichage formaté). Elle lève une `std::runtime_error` si le nom de fichier est vide.

3. La méthode `load(const std::string& filename)` doit simuler un chargement et retourner une `Party` reconstruite. Elle lève une `std::runtime_error` si le fichier "n'existe pas" (simuler par une condition sur le nom).
4. L'ensemble du système (Party, Characters, EventLog, SaveManager) doit fonctionner de concert dans un `main` qui : crée une session, la joue, la sauvegarde, gère les erreurs potentielles, puis tente de charger un fichier inexistant et gère l'exception.
5. Zéro `using namespace`, initialisation uniforme systématique, accès aux membres via getters `const`.

PARTIE 2 — APPRENTISSAGE AVANCÉ

Chapitre 9 — Gestion Mémoire Moderne : Smart Pointers

9.1 Le problème avec `new` et `delete`

En C, `malloc`/`free` forment une paire symétrique que vous gérez manuellement. En C++ pré-moderne, `new`/`delete` idem. Ces mécanismes présentent des risques fondamentaux :

```cpp
void old_style() {
    int *data = new int[100];

    process(data);            // Et si process() lève une exception ?

    delete[] data;            // Jamais atteint → fuite mémoire
}
```

Les pathologies classiques :

- **Memory leak** : `delete` oublié (surtout sur les chemins d'exception).
- **Double free** : `delete` appelé deux fois → undefined behavior.
- **Use after free** : pointeur utilisé après `delete` → undefined behavior.
- **Dangling pointer** : pointeur vers une zone libérée.

9.2 RAII : le principe fondateur

Resource Acquisition Is Initialization est l'idiome central du C++ moderne. L'idée : **lier la durée**

de vie d'une ressource à celle d'un objet. Quand l'objet est construit, il acquiert la ressource. Quand l'objet est détruit (automatiquement, à la fin de sa portée), il libère la ressource.

```cpp
// RAII appliqué manuellement
class ManagedArray {
    int *data_;
public:
    ManagedArray(size_t n) : data_{new int[n]} {}
    ~ManagedArray() { delete[] data_; }  // Libération garantie

    // Problème : que se passe-t-il si on copie cet objet ?
    // → Le Chapitre 10 y répond (Rule of 5)
};
```

Les **smart pointers** sont des implémentations RAII génériques pour les pointeurs.

9.3 `std::unique_ptr` : propriété exclusive

`std::unique_ptr<T>` est un **propriétaire exclusif** d'une ressource. Un seul `unique_ptr` peut posséder un objet à la fois. Quand le `unique_ptr` est détruit, il détruit automatiquement l'objet qu'il possède.

```cpp
#include <memory>

// Création : toujours via std::make_unique (C++14)
auto ptr = std::make_unique<Widget>(arg1, arg2);
// Équivalent à new Widget(arg1, arg2), mais exception-safe

// Utilisation : même syntaxe qu'un pointeur brut
ptr->method();
(*ptr).method();

// Destruction : automatique à la fin du scope
// Pas de delete nécessaire ni possible
```

La propriété exclusive est imposée par le compilateur :

```cpp
auto p1 = std::make_unique<Widget>();
auto p2 = p1;           // ERREUR DE COMPILATION : copie interdite
auto p2 = std::move(p1); // OK : transfert de propriété
// Après move : p1 est null, p2 possède l'objet
```

Sous le capot : `unique_ptr` a la même taille qu'un pointeur brut en mémoire (zero overhead). Le destructeur est injecté par le compilateur. Aucun coût à l'exécution par rapport à un `new`/`delete` manuel correctement utilisé.

```cpp
// Cas d'usage : fabrique (factory)
std::unique_ptr<Animal> make_animal(const std::string &type) {
    if (type == "dog") return std::make_unique<Dog>();
    if (type == "cat") return std::make_unique<Cat>();
    throw std::invalid_argument{"Unknown animal type"};
}
// Le caller possède le résultat et est responsable de sa durée de vie
```

9.4 `std::shared_ptr` : propriété partagée

`std::shared_ptr<T>` permet à **plusieurs propriétaires** de partager un objet. Il maintient un **compteur de références** (*reference count*) : le nombre de `shared_ptr` pointant vers l'objet. Quand le dernier `shared_ptr` est détruit, l'objet est détruit.

```cpp
auto sp1 = std::make_shared<Widget>();
// use_count() == 1

{
    auto sp2 = sp1;    // Copie : partage de propriété
    // use_count() == 2

    auto sp3 = sp1;
    // use_count() == 3
}
// sp2, sp3 détruits → use_count() == 1

// sp1 détruit → use_count() == 0 → Widget détruit
```

Coût de `shared_ptr` :

- Taille : **deux pointeurs** (un vers l'objet, un vers le bloc de contrôle).
- Bloc de contrôle : alloué séparément (sauf avec `make_shared` qui fusionne les deux allocations).
- Incrémentation/décrémentation du compteur : **atomique** (thread-safe) → coût non négligeable en contexte concurrent.

`std::weak_ptr` : observateur sans propriété

`weak_ptr` observe un objet géré par `shared_ptr` **sans en être propriétaire** (ne contribue pas au compteur). Il résout les **cycles de références** qui causent des fuites avec `shared_ptr` seul :

```cpp
struct Node {
    std::shared_ptr<Node> next;     // Propriété forte
    std::weak_ptr<Node> parent;     // Observation faible : pas de cycle
};

// Utiliser un weak_ptr : il faut d'abord le "promouvoir" en shared_ptr
if (auto locked = node->parent.lock()) {
    // locked est un shared_ptr valide : l'objet existe encore
    locked->method();
} else {
    // L'objet a été détruit
}
```

9.5 Règle d'usage en entreprise

Besoin	Solution
Un seul propriétaire	`std::unique_ptr`
Propriété partagée nécessaire	`std::shared_ptr`
Observer sans posséder	`std::weak_ptr`
Référence non-owning dans une fonction	Pointeur brut `T*` ou référence `T&`
`new`/`delete` raw	**Jamais**, sauf implémentation bas-niveau

> **Cahier des Charges — Exercice 9 : Graphe de Scène du Moteur**
> **Contexte métier :** Un moteur de jeu organise ses objets dans un graphe de scène (arbre hiérarchique). Chaque nœud peut avoir un parent et plusieurs enfants.
> **Règles métier à implémenter :**
> 1. Dans `engine`, créez une classe `SceneNode` représentant un nœud de scène avec : un nom, une liste d'enfants (chaque enfant est possédé exclusivement par son parent), et une référence non-propriétaire vers son parent.
> 2. La relation parent-enfant doit utiliser le smart pointer approprié à chaque direction : `unique_ptr` pour la relation de propriété descendante (parent vers enfants), et le mécanisme approprié pour la remontée non-propriétaire (enfant vers parent).

3. Implémentez une méthode `addChild(std::unique_ptr<SceneNode>)` qui transfère la propriété d'un nœud enfant vers le nœud courant, et définit correctement la relation vers le parent.
4. Implémentez une méthode `printTree(int depth) const` qui affiche récursivement l'arbre avec indentation représentant la profondeur.
5. Implémentez une méthode `findNode(const std::string& name) const` qui cherche un nœud par nom dans le sous-arbre (retourner un pointeur brut non-propriétaire ou nullptr).
6. Le `main` construit un arbre de scène d'au moins 3 niveaux de profondeur, l'affiche, cherche un nœud par nom, et vérifie que la destruction automatique (sortie de scope) ne génère aucune fuite. Aucun `new`/`delete` dans votre code client. Aucun `using namespace`.

Chapitre 10 — La Forme Canonique Orthodoxe (Rule of 3/5/0)

10.1 Le problème de la copie superficielle

Quand une classe gère une ressource (mémoire, fichier, socket, mutex…), les opérations de copie et de destruction générées par le compilateur par défaut sont **incorrectes** :

```cpp
class Buffer {
    char *data_;
    size_t size_;
public:
    Buffer(size_t n) : data_{new char[n]}, size_{n} {}
    // Pas de destructeur personnalisé, pas de constructeur de copie custom
};

Buffer b1{100};
Buffer b2 = b1;   // Copie SUPERFICIELLE : b2.data_ == b1.data_
// b1 et b2 pointent sur le même bloc mémoire !
// Quand b1 est détruit → delete[] data_
// Quand b2 est détruit → delete[] data_ (même pointeur) → DOUBLE FREE
```

10.2 La Rule of 3 (C++03)

Si vous définissez l'un de ces trois membres, vous devez définir les trois :

1. **Destructeur** (`~T()`)
2. **Constructeur de copie** (`T(const T&)`)

3. **Opérateur d'affectation de copie** (`T& operator=(const T&)`)

```cpp
class Buffer {
    char *data_;
    size_t size_;

public:
    // 1. Constructeur
    explicit Buffer(size_t n)
        : data_{new char[n]}, size_{n}
    { std::memset(data_, 0, size_); }

    // 2. Destructeur
    ~Buffer() { delete[] data_; }

    // 3. Constructeur de copie : copie PROFONDE
    Buffer(const Buffer &other)
        : data_{new char[other.size_]}, size_{other.size_}
    { std::memcpy(data_, other.data_, size_); }

    // 4. Opérateur d'affectation : idiome copy-and-swap
    Buffer &operator=(const Buffer &other) {
        if (this == &other) return *this;  // Auto-affectation
        Buffer temp{other};               // Copie via constructeur de copie
        std::swap(data_, temp.data_);     // Échange les ressources
        std::swap(size_, temp.size_);
        return *this;
        // temp (ancien this) est détruit ici
    }
};
```

10.3 La Rule of 5 (C++11) : Move Semantics

C++11 ajoute deux nouvelles opérations pour **transférer** une ressource sans la copier :

5. **Constructeur de déplacement** (`T(T&&)`)
6. **Opérateur d'affectation de déplacement** (`T& operator=(T&&)`)

Le `&&` désigne une **rvalue reference** : une référence à un objet temporaire ou dont on accepte de voler les ressources.

```cpp
```

```cpp
// Constructeur de déplacement : on "vole" les ressources de other
Buffer(Buffer &&other) noexcept
    : data_{other.data_}, size_{other.size_}
{
    other.data_ = nullptr;  // other ne peut plus libérer la ressource
    other.size_ = 0;
}

// Opérateur d'affectation de déplacement
Buffer &operator=(Buffer &&other) noexcept {
    if (this == &other) return *this;
    delete[] data_;          // Libérer l'ancienne ressource
    data_ = other.data_;     // Voler la ressource
    size_ = other.size_;
    other.data_ = nullptr;
    other.size_ = 0;
    return *this;
}
```

Pourquoi `noexcept` ? Les algorithmes de la stdlib (notamment `std::vector` lors de ses réallocations) n'utilisent le move que si la fonction est `noexcept`. Sans ce marqueur, votre type sera copié au lieu d'être déplacé dans ces contextes — performance dégradée.

10.4 La Rule of 0 : le meilleur choix

La meilleure façon d'appliquer la Rule of 5 est de… ne pas en avoir besoin. Si vos membres sont eux-mêmes des types RAII (`unique_ptr`, `vector`, `string`…), le compilateur génère automatiquement des opérations correctes.

```cpp
// Rule of 0 : aucune opération spéciale à définir
class Config {
    std::string name_;              // Gère sa propre mémoire
    std::vector<std::string> keys_; // Idem
    std::unique_ptr<Engine> engine_; // Gère sa propre ressource
    // Le compilateur génère correctement :
    // ~Config(), Config(Config&&), Config& operator=(Config&&)
    // Config(const Config&) est supprimé (à cause de unique_ptr)
};
```

Résumé :

Règle	Quand l'appliquer
Rule of 0	Toujours, si possible. Préférez des membres RAII.
Rule of 3	Si vous gérez une ressource et ne pouvez pas vous déplacer.
Rule of 5	Si vous gérez une ressource et voulez des performances optimales.

> **Cahier des Charges — Exercice 10 : Texture GPU Simulée**
> **Contexte métier :** Le moteur doit gérer des textures qui occupent de la mémoire "GPU" simulée (un bloc mémoire alloué dynamiquement).
> **Règles métier à implémenter :**
> 1. Dans `engine`, créez une classe `Texture` qui gère un tampon de données brutes (tableau de bytes alloué dynamiquement) représentant les pixels d'une texture. La classe stocke : les données, la largeur, la hauteur, et le nombre de canaux (ex : 4 pour RGBA).
> 2. Implémentez la Rule of 5 complète : constructeur principal, destructeur, constructeur de copie (copie profonde du tampon), opérateur d'affectation de copie (idiome copy-and-swap), constructeur de déplacement (`noexcept`), et opérateur d'affectation de déplacement (`noexcept`).
> 3. Chaque opération spéciale doit journaliser son exécution sur `std::cout` (ex: "Texture 'rock' : move constructor called") afin que vous puissiez observer le comportement lors des tests.
> 4. Créez une classe `TextureCache` qui stocke des textures via `std::unique_ptr` dans un `std::vector`. Elle expose une méthode pour ajouter une texture (transfert de propriété) et une méthode pour récupérer une référence constante par nom.
> 5. Le `main` doit démontrer : la création de textures, leur insertion dans le cache (observer le move), une tentative de copie d'une texture et son insertion dans le cache, et la destruction automatique des ressources en fin de scope. Comparer le nombre d'opérations de copie vs de déplacement dans la sortie.

Chapitre 11 — Conteneurs Avancés : Maps et Unordered Maps

11.1 `std::map` : dictionnaire ordonné

`std::map<K, V>` est un dictionnaire associatif qui maintient ses clés **triées par ordre croissant**. Il est implémenté comme un **arbre rouge-noir** (arbre binaire de recherche auto-équilibré).

```
std::map<std::string, int> scores

        "mario"(150)
       /           \
  "luigi"(90)   "peach"(200)
  /
"bowser"(300)
```

Complexité :

Opération	Complexité
Insertion, suppression, recherche	O(log n)
Itération ordonnée	O(n)
Minimum/Maximum	O(log n)

```cpp
```

```cpp
#include <map>

std::map<std::string, int> scores;

// Insertion
scores["alice"] = 100;                          // Opérateur [] : crée si absent
scores.insert({"bob", 200});                    // insert
scores.emplace("charlie", 150);                 // emplace : construction directe

// Accès
int s = scores.at("alice");   // Lève std::out_of_range si absent
// scores["unknown"] crée une entrée avec valeur 0 : PIÈGE fréquent !

// Recherche
auto it = scores.find("alice");
if (it != scores.end()) {
    // it->first == clé, it->second == valeur
    std::cout << it->first << ": " << it->second << "\n";
}

// Vérification d'existence (C++20)
if (scores.contains("bob")) { ... }

// Itération (toujours ordonnée alphabétiquement)
for (const auto &[key, value] : scores) {   // Structured bindings (C++17)
    std::cout << key << " → " << value << "\n";
}
```

11.2 `std::unordered_map` : dictionnaire par hachage

`std::unordered_map<K, V>` utilise une **table de hachage** (hash table). Pas d'ordre de parcours garanti, mais performances moyennes bien meilleures.

Complexité :

Opération	Cas moyen	Pire cas
Insertion, suppression, recherche	O(1) amorti	O(n) en cas de collision
Itération	O(n)	O(n)

```cpp
```

```cpp
#include <unordered_map>

std::unordered_map<std::string, int> registry;
// L'API est identique à std::map pour les opérations de base
registry["key"] = 42;
auto it = registry.find("key");
```

Sous le capot : chaque clé est hachée (`std::hash<K>`) pour obtenir un index dans un tableau de "buckets". En cas de collision (deux clés → même bucket), les éléments sont chaînés. Si le taux de remplissage (*load factor*) dépasse un seuil, la table est **rehashée** (réallocation + reinsertion de tout), ce qui invalide tous les itérateurs.

11.3 Quand choisir quoi

Critère	`std::map`	`std::unordered_map`
Itération ordonnée nécessaire	✅	❌
Recherche par range (`lower_bound`)	✅	❌
Performance brute O(1)	❌	✅
Type de clé sans `std::hash`	✅	Nécessite hash custom
Clé est un entier ou string standard	Indifférent	✅ (légèrement préféré)

11.4 Structured Bindings (C++17) avec les maps

```cpp
```

```
std::map<std::string, int> m{{"a", 1}, {"b", 2}};

// C++11/14 : verbeux
for (const auto &pair : m) {
    std::cout << pair.first << " " << pair.second;
}

// C++17 : structured bindings, bien plus lisible
for (const auto &[key, value] : m) {
    std::cout << key << " " << value;
}

// Insertion avec vérification (C++17)
auto [it, inserted] = m.insert({"c", 3});
if (inserted) { /* Nouvelle entrée */ }
else { /* Clé existante : it pointe dessus */ }
```

Cahier des Charges — Exercice 11 : Registre d'Assets du Moteur
Contexte métier : Le moteur doit centraliser tous ses assets (textures, sons, scripts) dans un registre global accessible par identifiant textuel.
Règles métier à implémenter :

1. Dans `engine`, créez une classe template `AssetRegistry<T>` qui associe des noms de string à des `std::unique_ptr<T>`. Utilisez le conteneur le plus approprié selon que l'accès ordonné est requis ou non (justifiez votre choix en commentaire).
2. La classe expose : une méthode `registerAsset(std::string name, std::unique_ptr<T> asset)` qui transfère la propriété (lever une exception si le nom existe déjà), une méthode `getAsset(const std::string& name) const` qui retourne un pointeur brut non-propriétaire (nullptr si absent), et une méthode `listAssets() const` qui affiche tous les noms enregistrés.
3. Instanciez le registre pour les `Texture` du chapitre précédent. Créez un `AssetRegistry<Texture>` et enregistrez plusieurs textures en transférant la propriété.
4. Implémentez une fonction `buildAssetReport(const AssetRegistry<Texture>&)` qui retourne une `std::map<std::string, std::string>` associant le nom de chaque texture à une description formatée (dimensions et canaux). Cette fonction illustre l'usage d'une map construite à la volée.
5. Le `main` intègre l'ensemble du système développé depuis le Chapitre 9 : scène, textures, registre, log. Il enregistre des textures, génère un rapport, tente d'enregistrer un doublon (exception), liste les assets, et récupère un asset par nom. Zéro `new`/`delete`, zéro `using namespace`.

Chapitre 12 — Notions Complémentaires Essentielles

12.1 Templates de fonctions et de classes

Les **templates** sont le mécanisme de généricité du C++. Ils permettent d'écrire du code paramétré par des types, résolu **à la compilation**.

```cpp
// Template de fonction
template<typename T>
T clamp(T value, T min_val, T max_val) {
    if (value < min_val) return min_val;
    if (value > max_val) return max_val;
    return value;
}

// Utilisation : le compilateur instancie une version par type utilisé
int a = clamp(15, 0, 10);        // clamp<int>
float b = clamp(3.5f, 0.0f, 1.0f); // clamp<float>
```

```cpp
// Template de classe
template<typename T>
class Stack {
    std::vector<T> data_;
public:
    void push(const T &val) { data_.push_back(val); }
    void push(T &&val) { data_.push_back(std::move(val)); }
    T pop() {
        if (data_.empty()) throw std::underflow_error{"Stack is empty"};
        T val = std::move(data_.back());
        data_.pop_back();
        return val;
    }
    bool empty() const { return data_.empty(); }
    size_t size() const { return data_.size(); }
};

Stack<int> int_stack;
Stack<std::string> string_stack;
```

Les enums C classiques ont deux problèmes : leurs valeurs polluent le namespace englobant et elles se convertissent implicitement en entiers.

```cpp
// C / C++ ancien : DANGEREUX
enum Color { RED, GREEN, BLUE };
enum Status { RED, ACTIVE };  // ERREUR : RED redéfini !
int c = RED;                  // Conversion implicite silencieuse

// C++11 : enum class — typé et scoped
enum class Color { Red, Green, Blue };
enum class Status { Red, Active };    // OK : pas de conflit

Color c = Color::Red;
// int x = Color::Red;               // ERREUR : pas de conversion implicite
int x = static_cast<int>(Color::Red); // OK : conversion explicite
```

12.3 `constexpr` : calcul à la compilation

`constexpr` indique qu'une valeur ou une fonction peut être évaluée **à la compilation** si ses arguments sont connus à la compilation.

```cpp
constexpr int factorial(int n) {
    return n <= 1 ? 1 : n * factorial(n - 1);
}

constexpr int f5 = factorial(5);  // Calculé à la compilation : 120
// Équivalent à : constexpr int f5 = 120;

// Tableaux de taille fixe calculée à la compilation
constexpr size_t TILE_COUNT = 16 * 16;
std::array<int, TILE_COUNT> tilemap;  // Taille connue à la compilation
```

En C++20, `constexpr` est très largement étendu : des fonctions virtuelles, des allocations dynamiques, et des algorithmes de la stdlib peuvent être `constexpr`.

12.4 `std::optional` : valeur ou absence

`std::optional<T>` représente une valeur qui peut être **présente ou absente**, sans recourir à un pointeur nullable ou à une valeur sentinelle (-1, nullptr…).

```cpp
```

```cpp
#include <optional>

std::optional<std::string> find_player(int id) {
    if (id == 1) return "Alice";
    return std::nullopt;  // Absence explicite
}

auto name = find_player(1);
if (name.has_value()) {         // ou if (name)
    std::cout << *name << "\n"; // ou name.value()
}

// Valeur par défaut si absent
std::string player = find_player(99).value_or("Unknown");
```

12.5 Lambdas et `std::function`

Une **lambda** est une **fonction anonyme** définie inline, pouvant capturer des variables locales.

```cpp
// Syntaxe : [capture](params) -> return_type { body }
auto add = [](int a, int b) { return a + b; };
int result = add(3, 4);  // 7

// Capture par valeur
int threshold = 10;
auto is_above = [threshold](int val) { return val > threshold; };

// Capture par référence
int counter = 0;
auto increment = [&counter]() { ++counter; };
increment();  // counter == 1

// Capture universelle (déconseillé en production : cache les dépendances)
auto all_by_value = [=]() { ... };
auto all_by_ref   = [&]() { ... };
```

```cpp
```

```cpp
// Utilisation avec les algorithmes de la stdlib
std::vector<int> v{5, 1, 8, 3, 9, 2};

std::sort(v.begin(), v.end(), [](int a, int b) { return a > b; }); // Tri décroissan

auto it = std::find_if(v.begin(), v.end(), [](int x) { return x > 7; });

// std::function : stocker une lambda ou tout callable
std::function<void(const std::string&)> logger;
logger = [](const std::string &msg) { std::cout << "[LOG] " << msg << "\n"; };
```

12.6 Algorithmes de la stdlib `<algorithm>`

La stdlib C++ fournit des algorithmes génériques qui opèrent sur des ranges (paires itérateur begin/end) :

```cpp
#include <algorithm>
#include <numeric>

std::vector<int> v{3, 1, 4, 1, 5, 9};

std::sort(v.begin(), v.end());                    // Tri croissant
std::reverse(v.begin(), v.end());                 // Inversion
auto max = std::max_element(v.begin(), v.end());  // Itérateur vers le max
int sum = std::accumulate(v.begin(), v.end(), 0); // Somme

// Avec prédicats lambda
std::sort(v.begin(), v.end(), std::greater<int>{}); // Tri décroissant

int count = std::count_if(v.begin(), v.end(),
    [](int x) { return x % 2 == 0; });  // Compter les pairs

// Copie conditionnelle
std::vector<int> odds;
std::copy_if(v.begin(), v.end(), std::back_inserter(odds),
    [](int x) { return x % 2 != 0; });
```

C++20 — Ranges : Une nouvelle approche qui permet de chaîner les opérations sans nommer les itérateurs :

```cpp
```

```
#include <ranges>

// Filtrage et transformation pipeline
auto result = v
    | std::views::filter([](int x) { return x > 3; })
    | std::views::transform([](int x) { return x * x; });

for (int x : result) { std::cout << x << "\n"; }
```

Cahier des Charges — Exercice 12 : Moteur de Recherche d'Assets (Synthèse Finale)
Contexte métier : Cet exercice est la **synthèse finale** de l'intégralité du cours. Vous implémentez un sous-système complet du moteur de jeu.
Règles métier à implémenter :

1. Créez un template de classe `EventQueue<T>` dans `engine` qui stocke des événements de type T en attente de traitement. Elle doit utiliser un `std::vector` en interne et exposer : `push(T)`, `processAll(std::function<void(const T&)>)` (vide la queue en appelant le handler sur chaque élément), et `size() const`.
2. Définissez un type `AssetEvent` (struct ou classe) avec : un enum class `EventType` (Loaded, Unloaded, Failed), un nom d'asset, et un message optionnel (`std::optional<std::string>`).
3. Modifiez `AssetRegistry<T>` pour qu'il pousse un `AssetEvent` dans une `EventQueue<AssetEvent>` lors de chaque enregistrement ou suppression d'asset (la queue est fournie à la construction du registry).
4. Implémentez une fonction libre `filterAssets` qui prend un `AssetRegistry<Texture>` et un prédicat (`std::function<bool(const std::string&, const Texture*)>`) et retourne un `std::vector<std::string>` des noms d'assets correspondants.
5. Dans `engine`, ajoutez une constante `constexpr` représentant la version du moteur (major, minor, patch), et affichez-la au démarrage.
6. Le `main` assemble l'intégralité du système : version du moteur, EventLog, SceneNode, Texture, AssetRegistry, EventQueue. Il enregistre des textures, déclenche des events, les traite via un handler lambda, filtre les assets via prédicat, et affiche un rapport final. La sortie doit démontrer : zero fuite mémoire (tous les destructeurs se déclenchent à la fin du scope), zero `new`/`delete` dans le code client, zero `using namespace`.

ANNEXES

Annexe A — Référence Rapide : Choisir son Conteneur

Besoin	Conteneur	Complexité accès
Tableau dynamique, accès indexé	`std::vector<T>`	O(1)
Insertion/suppression en tête et queue	`std::deque<T>`	O(1)
Liste doublement chaînée	`std::list<T>`	O(n) accès, O(1) insert
Dictionnaire ordonné	`std::map<K,V>`	O(log n)
Dictionnaire non ordonné (rapide)	`std::unordered_map<K,V>`	O(1) moyen
Ensemble ordonné (clés uniques)	`std::set<T>`	O(log n)
File de priorité	`std::priority_queue<T>`	O(log n) push/pop
Tableau taille fixe compile-time	`std::array<T,N>`	O(1)

Annexe B — Référence Rapide : Smart Pointers

cpp

```cpp
// unique_ptr
auto u = std::make_unique<T>(args...);
auto u2 = std::move(u);          // Transfert de propriété
T *raw = u.get();                // Accès au pointeur brut (sans céder la propriété)
u.reset();                       // Libère et met à nullptr

// shared_ptr
auto s = std::make_shared<T>(args...);
auto s2 = s;                     // Copie : partage de propriété
long count = s.use_count();      // Nombre de shared_ptr propriétaires

// weak_ptr
std::weak_ptr<T> w = s;
if (auto locked = w.lock()) {    // Promotion : nullptr si objet détruit
    locked->method();
}
```

Annexe C — Compilation : Drapeaux Essentiels

```bash
# Compiler avec C++20, warnings maximaux, optimisations de debug
g++ -std=c++20 -Wall -Wextra -Wpedantic -g -o output main.cpp

# Build de production
g++ -std=c++20 -O2 -DNDEBUG -o output main.cpp

# Avec sanitizers (détecter les erreurs mémoire et UB en développement)
g++ -std=c++20 -Wall -g -fsanitize=address,undefined -o output main.cpp

# Clang (souvent de meilleurs messages d'erreur)
clang++ -std=c++20 -Wall -Wextra -g -o output main.cpp
```

Annexe D — Checklist Code Review C++ Moderne

- [ ] Zéro `new` / `delete` raw dans le code client
- [ ] Zéro `using namespace std;` dans les headers
- [ ] Tous les smart pointers créés via `make_unique` / `make_shared`
- [ ] Méthodes non-modifiantes marquées `const`
- [ ] Arguments de grande taille passés par `const &`
- [ ] Constructeurs de move marqués `noexcept`
- [ ] Rule of 0 préférée (membres RAII), Rule of 5 si gestion manuelle de ressource
- [ ] Exceptions levées pour les erreurs de précondition, pas de codes de retour
- [ ] `enum class` au lieu de `enum`
- [ ] Initialisation uniforme `{}` systématique
- [ ] `[[nodiscard]]` sur les fonctions dont le retour ne doit pas être ignoré
- [ ] Pas de `nullptr` déréférencé (vérifier les pointeurs bruts non-owning)